

# Software Verification and Validation

Part Time Lecturer  
Talimul Bari Shreshtho

Islamic University of Technology

June 19, 2026



# Table of Contents

## Definition

Software Testing is the process of executing a software system to identify defects and verify that it satisfies specified requirements.

## Objectives of Testing

- Detect defects and failures
- Verify functional requirements
- Validate user expectations
- Improve software quality
- Increase confidence in the system

# Why Do We Test?

Software systems are complex and errors can be introduced during:

- Requirements specification
- Design
- Coding
- Integration
- Maintenance

**Testing helps to**

- Find defects before deployment
- Reduce risk and cost of failures
- Improve reliability and quality
- Ensure customer satisfaction

# An Important Truth About Testing

*“Testing can show the presence of defects, but not their absence.”*

Even if all tests pass:

- Some defects may still exist.
- Not all inputs and scenarios can be tested.
- Testing increases confidence but cannot prove correctness.

## **Example**

Testing a login system with 100 test cases does not guarantee that the 101st scenario will work correctly.

# A Practical Testing Strategy

When testing a feature, proceed in the following order:

- ① Test the Happy Path
- ② Test Alternate Paths
- ③ Test Error Paths
- ④ Test Data Variations
- ⑤ Test Boundary Conditions
- ⑥ Test Previously Found Defects

## **Example: ATM Withdrawal**

- Happy Path: Valid withdrawal
- Alternate Path: Different account types
- Error Path: Insufficient balance
- Boundary: Balance = withdrawal amount
- Defect Regression: Previously fixed bug

# Happy Path, Alternate Path, and Error Path

**Happy Path** Normal expected behavior.

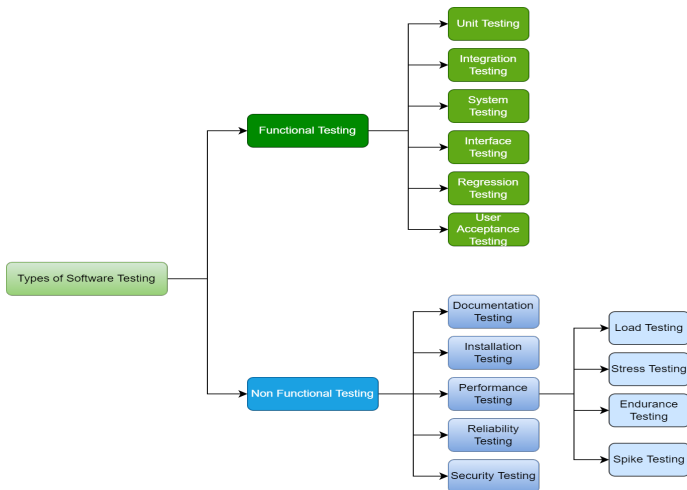
**Alternate Path** Valid variations of normal behavior.

**Error Path** Invalid inputs, failures, and exceptional situations.

## **Example: User Login**

- Happy Path: Correct username and password
- Alternate Path: Login using email address
- Error Path: Invalid password

# Types of Software Testing



## Definition

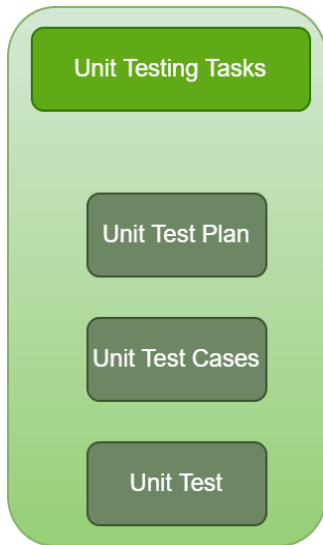
Unit Testing is the process of testing the smallest testable component (unit) of a software system in isolation to verify its functional correctness.

## Characteristics

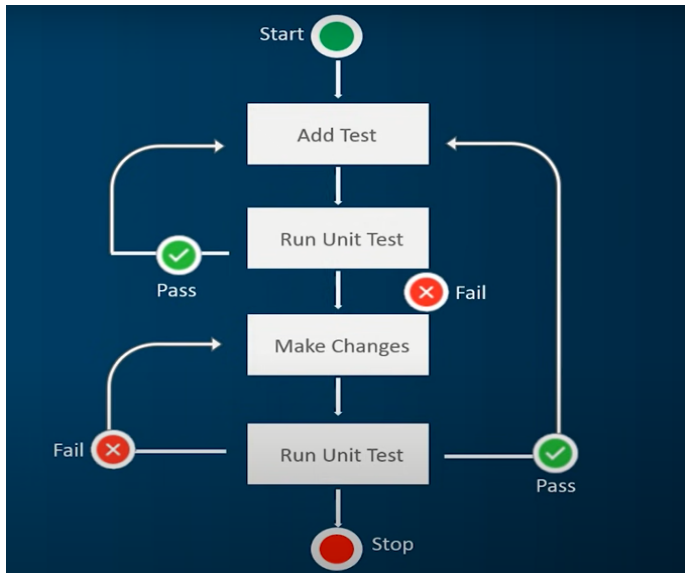
- Focuses on individual functions, methods, classes, or modules
- Performed before Integration Testing
- Helps detect defects early in development
- Usually automated using testing frameworks
- Uses test stubs, drivers, and mocks when needed

## Goal

*Verify that each unit behaves as expected for valid, invalid, and boundary inputs.*



# Unit Testing Algorithm



# Benefits of Unit Testing

- Makes coding practice more agile.
- Improves the quality of the code.
- Helps find software bugs early.
- Facilitates changes and simplifies integration.
- Provides documentation to developers.
- Makes debugging easier and quicker.
- Helps create better software designs.
- Reduces overall testing cost.

# White-box vs Black-box Testing

## **White-box Testing (Structural Testing)**

Covers the internal logic of the program:

- Statement coverage
- Decision / Branch coverage
- Condition coverage
- Decision-condition coverage
- Multiple condition coverage

## **Black-box Testing (Functional Testing)**

Covers functional specifications without internal code knowledge:

- Boundary Value Analysis (BVA)
- Equivalence Class Testing
- State Transition Table-based Testing
- Decision Table-based Testing
- Cause-Effect Graphing

# Boundary Value Analysis (BVA)

**Idea:** Test cases are designed using boundary values because most errors occur at the edges of input domains.

**Definition:** BVA is a black-box testing technique that focuses on testing:

- Minimum value
- Maximum value
- Values just above/below boundaries

**Example:** If  $A \in [10, 255]$ , then test:

- 9, 10, 11
- 254, 255, 256

# Boundary Value Checking (BVC)

**Method:** One variable is tested at boundary values while others are kept at nominal values.

**Boundary values used:**

- Min
- Min + 1
- Max
- Max - 1
- Nominal

**Result:** For  $n$  variables:

$4n + 1$  test cases

**Idea:** Extend BVC by including values outside the valid range.

**Additional test cases:**

- Min - 1
- Max + 1

**Result:** For  $n$  variables:

$6n + 1$  test cases

**Idea:** Test all combinations of boundary values for multiple variables.

**Includes:**

- All boundary combinations
- Multiple variables at boundary simultaneously

**Result:** For  $n$  variables:

$5^n$  test cases

# Boundary Value Analysis (BVC)

**Basic Idea:** Test one variable at the boundary, others at nominal values.

**Test Cases (A, B):**

| TC | A Value | B Value |
|----|---------|---------|
| 1  | A_nom   | B_nom   |
| 2  | A_min   | B_nom   |
| 3  | A_min+  | B_nom   |
| 4  | A_max   | B_nom   |
| 5  | A_max-  | B_nom   |
| 6  | A_nom   | B_min   |
| 7  | A_nom   | B_min+  |
| 8  | A_nom   | B_max   |
| 9  | A_nom   | B_max-  |

**Idea:** Extend BVC by adding invalid boundary values.

**Additional Test Cases:**

| TC | A Value | B Value |
|----|---------|---------|
| 10 | A_max+  | B_nom   |
| 11 | A_min-  | B_nom   |
| 12 | A_nom   | B_max+  |
| 13 | A_nom   | B_min-  |

**Result:** Detects robustness against invalid inputs.

# Worst-Case Testing

**Idea:** Combine all boundary values of multiple variables.

**Key Concept:** All combinations of boundary values are tested.

**Example (Partial List):**

| TC | A Value | B Value |
|----|---------|---------|
| 14 | A_min   | B_min   |
| 15 | A_min+  | B_min   |
| 16 | A_max   | B_max   |
| 17 | A_max-  | B_max-  |
| 18 | A_min   | B_max   |
| 19 | A_max   | B_min   |

**Note:** For  $n$  variables, worst-case produces  $5^n$  test cases.

# Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`